Multi-Heuristic A* Campus Navigation and Optimal Route Finder

BCSE306L - Artificial Intelligence
Module 2: Problem Solving based on Searching

Implementation and experiment report
Prepared from the supplied course proposal and campus map

Team member	Registration number	Assigned responsibility
Nimisha	24BCT0188	A* engine and state tracking
Rohan	24BCE0985	Custom binary min-heap
Ishaan	24BDS0243	UCS and Greedy baselines
Rahul	24BCE3006	Heuristics and correctness reasoning
Tanishq	24BCE0980	Graph modelling and experiments

1. Problem statement and motivation

Given a weighted graph of campus road access points and pedestrian links, find the least-cost route between any selected origin and destination. Compare how heuristic choice changes node expansions, runtime and path optimality.

A student moving between an academic block, a gate and a hostel may face several possible routes. A graph captures this decision: vertices describe meaningful locations, edges describe allowed movements, and weights describe estimated walking distance. The graph is small enough to inspect, while the lake, railway and subway connections create useful routing constraints.

The project implements A* and its supporting heap explicitly. Uniform Cost Search supplies a shortest-cost baseline. Greedy Best-First Search shows the effect of prioritising apparent closeness without accounting for the distance already travelled. Euclidean and Manhattan variants demonstrate why heuristic validity matters.

Scope: this is a teaching model built by manually digitising the supplied image. The image has no surveyed scale. All distances are labelled estimated model metres. Real access permissions and road connections remain to be checked on campus. OpenAI Codex assistance is declared on page 10.

2. Formal AI formulation

Let G = (V, E) be a finite directed graph. Each vertex v has planar reference-image coordinates (x_v, y_v). Each directed edge (u,v) has a finite positive cost c(u,v). The default graph contains 52 vertices and 126 directed arcs. Each of its 63 modelled roads currently permits traversal in both directions.

Search component	Formal definition
State space	S = V. A state identifies a road access point or junction.
Initial state	s_0 = s, the user-selected origin.
Actions	A(v) = { traverse(v,u) : (v,u) is an available arc in E }.
Transition	Result(v, traverse(v,u)) = u. The transition is deterministic.
Goal test	Goal(v) is true exactly when v = t, the selected destination.
Step cost	c(v,u) = scale times traced road-polyline length, rounded upward.
Path cost	g(v_k) = sum of c(v_i,v_(i+1)) along the discovered route.
Objective	Minimise the sum of edge costs from s to t.

Inputs, outputs and failure

The input consists of a graph, a start ID and a goal ID. Optional road closures remove directed arcs before search. The output is an ordered path, its accumulated cost, an expansion count and search latency. An unknown node produces an input error. An exhausted frontier produces an empty route with cost None. If start equals goal, the result is a one-node route with cost zero and no expansions.

Expansion means generating a valid popped node's successors. The final goal pop does not generate successors and therefore does not count as an expansion. Stale heap entries also do not count. If an inconsistent heuristic forces a node to reopen, that later successor generation counts again.

The objective is distance alone. Expansions and runtime quantify computational effort, rather than adding penalties to route cost. Different optimal paths may have the same total distance. The program does not require their vertex sequences to match, only that the path be valid and its cost equal the independently verified optimum.

3. Dataset and campus model

The graph stores 30 selectable landmarks and 22 connecting junctions. Locations include Main Gate, Library, MGR, SMV, Technology Tower, SJT, PRP, MGB, hostel access points and Gates 2A, 3, 5 and 11. Coordinates refer to a 1882 x 1344 rendering of the supplied campus image. They denote approximate road access points, not building interiors.

Dataset component	Representation
Vertices	52 named road access points and junctions
Roads	63 manually traced road polylines
Directed arcs	126 arcs: two directions for each default road
Reference coordinates	x and y in a 1882 x 1344 reference image
Edge weight	Traced segment lengths times a common scale
Default scale	1 estimated model metre per reference pixel
Railway crossings	Explicit S1, S2, S3 and S4 subway links
Lake barrier	No edge crosses the lake

Road bends are stored as intermediate coordinates. Edge weight is the sum of straight segments along that polyline, multiplied by the chosen scale. Rounding upward to 0.001 model metre preserves the lower bound required by Euclidean A*. The generator rejects non-finite or non-positive costs and edges shorter than endpoint displacement.

The map has no surveyed scale. Model metres are illustrative units, not verified physical distances. Editing scale changes both costs and heuristics. Road access, hostel permissions and current subway availability need an on-campus check. Algorithm correctness alone cannot establish that a road is accessible.

The editable dataset source is step01_graph_Tanishq.py. The generated campus_graph.json exports all points, arcs, road bends, scale and units. This version provides routes and comparison evidence as terminal text and CSV tables. No generated route image or chart is required.

4. Algorithm description and pseudocode

A* ranks frontier entries by f(n) = g(n) + h(n). The heap returns the smallest priority. g is the cheapest discovered route to a node so far, while h estimates the remaining distance. A parent dictionary supports route reconstruction. The goal test follows a valid minimum-priority pop.

```
A_STAR(graph, start, goal, h):
  best_g[start] = 0; parent = empty
  heap.push(h(start), sequence=0, start, g=0)
  while heap is not empty:
    f, sequence, node, popped_g = heap.pop()
    if popped_g > best_g[node]: continue
    if node expanded at an equal/better cost: continue
    if node == goal: return reconstruct(parent), popped_g
    record expansion at popped_g
    for neighbor, edge_cost in outgoing_edges(node):
      candidate = popped_g + edge_cost
      if candidate < best_g.get(neighbor, infinity):
        best_g[neighbor] = candidate
        parent[neighbor] = node
        sequence += 1
        heap.push(candidate+h(neighbor), sequence,
                  neighbor, candidate)
  return no route
```

AI decisions and frontier handling

Two operations show the reasoning directly: removing the entry with minimum g+h, and accepting a neighbor only when candidate improves its recorded route cost. These choices focus search while preserving cheaper discovered routes. Returning when the goal first becomes a neighbor would be incorrect because another frontier route could reach it more cheaply.

The implementation uses lazy decrease-key. A cheaper route adds a new entry instead of locating an old entry inside the heap. When the older entry eventually appears, its recorded cost exceeds best_g and the algorithm skips it. Previously expanded nodes can reopen at an improved cost. This handles admissible but inconsistent heuristics on general graphs.

Comparison algorithms

UCS is the same search with h=0, so it uses priority g and needs no duplicate engine. Greedy uses priority h only and records a node on first discovery. It is complete on this finite graph with its discovered set, but it has no shortest-cost guarantee. Its path is checked for legal edges and correct accumulated cost rather than asserted optimal.

5. Heuristics and correctness reasoning

Heuristic	Definition, with scale k	Status in the model
Zero	h_0 = 0	Admissible and consistent
Euclidean	h_E = k * sqrt(dx^2 + dy^2)	Admissible and consistent
Scaled Manhattan	h_S = k * (|dx|+|dy|) / sqrt(2)	Admissible and consistent
Raw Manhattan	h_M = k * (|dx|+|dy|)	May overestimate

Admissibility means h(n) never exceeds the cheapest remaining cost h*(n). **Consistency** means h(u) <= c(u,v) + h(v) for every arc, with h(goal)=0. A consistent estimate bounds the total cost of any remaining route by repeated application of that inequality.

Euclidean proof

Each edge costs at least its endpoint Euclidean displacement. Triangle inequality gives d_E(u,t) <= d_E(u,v) + d_E(v,t). Multiplying by the common scale and replacing the first term by its edge-cost upper bound gives h_E(u) <= c(u,v) + h_E(v). Thus Euclidean is consistent and admissible in the validated model.

Scaled Manhattan proof

For any displacement, (|dx|+|dy|)/sqrt(2) <= sqrt(dx^2+dy^2). Scaled Manhattan is itself a metric, so its triangle inequality also applies. Its endpoint distance is no greater than the Euclidean endpoint distance, which is no greater than edge cost. Therefore h_S(u) <= c(u,v)+h_S(v). This is a valid but generally weaker bound than Euclidean.

Why raw Manhattan is experimental

A diagonal path can be shorter than |dx|+|dy|. Consider S=(0,0), A=(1,1), G=(2,2), with edges S-A=1.415, A-G=1.415 and S-G=3. The shortest route costs 2.83. Raw Manhattan assigns A priority 1.415+2=3.415, so A* pops G at cost 3 first. This is a concrete loss of optimality, despite every edge satisfying the Euclidean lower bound.

The original proposal included raw Manhattan. The project retains it to measure this failure and adds scaled Manhattan as a safe comparison. Reopening does not make an overestimating heuristic admissible. Euclidean A* is the default route finder. Its guarantee applies to the graph under the stated assumptions, rather than to unverified real-world roads.

6. Implementation notes and role division

File in execution order	Purpose
step01_graph_Tanishq.py	Dataset generation, graph validation, scale and closures
step02_heap_Rohan.py	Explicit binary min-heap push/pop and sift operations
step03_heuristics_Rahul.py	Four estimates and consistency audit
step04_astar_Nimisha.py	A*, best_g, parents, stale entries, reopening and trace
step05_baselines_Ishaan.py	UCS using h=0 and separate Greedy search
step06_run_Tanishq.py	Input prompts, demo, paired comparison tables and verification

There are six project Python files. The final runner imports the earlier modules, so one command executes the project. Running it normally lists landmark IDs and asks for Start and Goal. It accepts repeated route queries until the user types q. Explicit start/goal arguments, the benchmark and verification commands run without keyboard prompts.

The graph uses an adjacency dictionary. Heap entries are tuples (priority, sequence, node, discovered_cost). Insertion sequence gives deterministic FIFO tie-breaking. best_g and parent are dictionaries. Expanded costs distinguish stale entries from useful reopenings. Route reconstruction follows parent links backwards and reverses the list.

For a consistent heuristic, this lazy heap uses roughly O((V+E) log E) time and O(V+E) auxiliary memory. An inconsistent heuristic can cause additional expansions. The separate Floyd-Warshall verification oracle uses O(V^3) time and O(V^2) memory, appropriate for this small graph but not used as the navigation algorithm.

The entire executable project now requires only Python 3.9+ and its standard library. Comparison tables appear in the terminal and in CSV files. There are no plotting or image-loading dependencies. The supplied campus image remains a reference for the manually digitised graph.

Positive-cost validation avoids negative-edge assumptions. Same-node routes, unknown IDs, unreachable goals and road closures have defined behavior. Closures remove both directions of a modelled road. Search itself reads outgoing arcs and therefore also supports directed graphs. The assigned files identify team review responsibilities for an AI-assisted draft.

7. Experimental method and results

A seeded sample selects 50 distinct ordered pairs from the 30 landmarks, excluding identical endpoints. Seed 42 makes selection reproducible. Every algorithm receives the same pairs. Each query has one untimed warm-up and 21 timed searches. Reported latency is the mean of 50 per-query median runtimes.

Timing includes search, reconstruction and trace creation. File writing and verification are outside the timer. Costs are compared with independent Floyd-Warshall distances, using an absolute tolerance of 1e-7. Timing varies by machine. The supplied experiment.json records the exact settings and machine information.

Algorithm	Optimal /50	Expansions	Mean ms	Gap %
UCS	50	30.720	0.034938	0.000
Greedy Euclidean	29	8.640	0.012857	8.259
A* Euclidean	50	13.780	0.023924	0.000
A* Scaled Manhattan	50	16.480	0.030731	0.000
A* Raw Manhattan	42	10.520	0.018959	0.160

All 50 cases reached the goal. UCS, Euclidean A* and scaled Manhattan A* found the optimum every time. Greedy missed the optimum on 21 pairs, and raw Manhattan A* missed it on eight. These are measured outcomes, not assumed performance claims.

Examples of sampled test cases

Case	Origin	Destination
1	TT	MH_OFFICE
2	SMV	MGB
3	MAIN_GATE	PRP
4	PRP	LH_AB
5	GATE2A	MH_N

benchmark_queries.csv contains all 250 rows: 50 pairs times five methods. Each row records the origin, destination, returned and oracle costs, expansions, median runtime, reachability, optimality and excess-cost percentage. benchmark_summary.csv provides the aggregates above. The professor permits a comparison table or plot, so the tables supply the comparison evidence.

8. Analysis and worked demonstration

Euclidean A* averaged 13.78 expansions versus UCS's 30.72, a reduction of 55.1%, while preserving shortest costs. Its lower bound focuses exploration. Scaled Manhattan averaged 16.48 expansions. It is valid but never stronger than Euclidean for these coordinates.

Greedy used only 8.64 expansions on average, but missed the optimum in 21 of 50 queries. Its mean excess cost was 8.259%, with a worst excess of 70.018%. Raw Manhattan averaged 10.52 expansions and missed the optimum in eight queries. Its mean excess was 0.160%, with a worst excess of 4.277%. Fewer expansions alone do not establish a better route finder.

All measured searches finish in fractions of a millisecond on this machine. Such small runtime differences should not be generalised to other hardware or large graphs. Expansion counts are stable for fixed data, adjacency order, heuristic and tie-breaking. Cost optimality is checked independently rather than inferred from speed.

User input demonstration

```
python3 step06_run_Tanishq.py
Start: MAIN_GATE
Goal: SJT
```

The default Euclidean A* method returns 1384.495 estimated model metres and 24 expansions for this example. It follows the route below. Other start and goal IDs are accepted at the prompts.

```
MAIN_GATE -> MGR -> WEST_CENTER -> WEST_MID
-> WOODYS_S -> S3_S -> TT_N -> LAKE_NW
-> S2_S -> LH_C -> LH_OFFICE -> SJT_J -> SJT
```

The route stays south of the railway and follows the road north of the lake. Its bends explain why straight-line endpoint distance is only a lower bound. search_trace.csv records every valid pop's g, h and f, including the goal. The same-node and closed-road demonstrations provide additional evidence beyond the fixed route.

What worked and what failed

The safe geometric heuristics reduced exploration while retaining verified shortest costs. The raw Manhattan assumption failed because the graph allows diagonal segments. The implementation retains that variant as an honest experimental comparison. Graph accuracy remains dependent on manual digitisation and field checks.

9. Verification, limitations and improvements

The verification command runs 13,520 campus searches: all 2,704 ordered graph-node pairs, including identical nodes, through five methods. It checks returned endpoints, edge legality and summed cost. UCS and both safe A* methods must match an independently implemented Floyd-Warshall oracle. Non-optimal methods are measured rather than incorrectly forced to pass an optimality assertion.

Verification evidence	Result
Campus search checks	13,520 checks passed
Random directed graphs	980 UCS/oracle checks passed on 20 seven-node graphs
Safe heuristic audit	Zero admissibility and consistency violations
Raw Manhattan audit	1,236 inadmissible node/goal pairs; 1,141 inconsistent arc/goal triples
Exhaustive non-optimal routes	302 raw Manhattan pairs and 852 Greedy pairs exceed the optimum
Special cases	Heap ordering, mixed push/pop, invalid IDs, directed edges, unreachable, same node, reopening, closure and diagonal failure

A constructed admissible but inconsistent heuristic forces A to expand, then reopen with a cheaper route. This verifies a behavior absent from the default consistent-heuristic runs. Random directed graphs check asymmetric edges, dead ends and improved routes. Heap tests compare custom operations with independently sorted values. Python assertions must remain enabled when using --verify.

Limitations

The map provides no surveyed metric scale or current path-access rules. Points describe approximate road entrances. The model omits many buildings and internal walkways. It does not model stairs, wheelchair access, crowds, time-dependent closures, GPS or image-based obstacle detection. Collision-free means following supplied graph links around modelled barriers. It is not an implemented geometric collision detector.

Next improvements

Walk the selected links and record permitted access. Measure several routes to calibrate a common scale and refine coordinates. Replace approximate polylines with confirmed paths and add missing connections only after inspection. Revalidate the edge lower bound and rerun the full correctness checks after edits. Additional costs such as travel time require a corresponding valid time heuristic, rather than blindly reusing geometric distance.

10. References, disclosure and use

References

1. Supplied team course proposal, Multi-Heuristic A* Campus Navigation and Optimal Route Finder, September 11, 2026. Used for title, scope, IDs and team roles.

2. Office of Students' Welfare, VIT Vellore, supplied campus map. Used for approximate manual digitisation. Survey scale and current access are unspecified.

3. Nancy Lydia R, supplied BCSE306L project-guidelines email, September 7, 2026. Used for required implementation, comparison, report and viva.

4. UC Berkeley CS188, Informed Search: https://inst.eecs.berkeley.edu/~cs188/textbook/search/informed.html

5. Russell and Norvig, AIMA informed-search slides: https://aima.cs.berkeley.edu/slides-pdf/chapter04a.pdf

External AI tool declaration

OpenAI Codex drafted substantial portions of the Python implementation, approximate map digitisation, verification cases, report and presentation. No external AI model or API performs routing during execution. Search, heuristic formulas and the binary heap are explicit local Python code. The generated safe methods were checked against an independent shortest-path oracle.

Named files identify the proposal's assigned review responsibilities, not independent authorship. Team members must understand, correct and validate this draft before submission and retain the disclosure. No on-campus survey or independent student validation is claimed.

Reproduction and demonstration

```
python3 step06_run_Tanishq.py --verify --benchmark
python3 step06_run_Tanishq.py
```

For an eight-minute demonstration, explain the problem and data assumptions, give the search formulation, enter a professor-selected route, point to the heap and cheaper-route decisions, explain the comparison table, and demonstrate closures and failure cases. Finish with limitations and disclosure. README.md contains the exact commands.

For each individual viva, explain g, h and f, admissibility versus consistency, the pop-time goal test, UCS as h=0, lazy entries and reopening, and the raw Manhattan counterexample. The algorithm and comparison tables provide the evidence. Python alone is sufficient to run the project.